

יחידה 5.3 – מחרוזות

בודק בטיחות לחשבון

`P@ss123!`

הבעיה

אתר בודק סיסמה וכתובת דוא״ל לפני שפותחים חשבון. משתמשים בנתונים בדויים בלבד.

- סיסמה באורך 6 תווים לפחות, אחרת שואלים שוב.
- אזהרה אם היא מתחילה ב-`pass` או מסתיימת ב-`123`.
- ספירת אותיות, ספרות, אותיות גדולות וסימנים.
- ציון מ-0 עד 5 ודירוג. מחרוזות חדשות. הצעה לסיסמה חזקה.
- בדיקת כתובת דוא״ל, ושם המשתמש והדומיין שלה.

דוגמת הרצה

```
Password: abc
Too short, try again
Password: Pass123
Weak pattern found
Letters: 4
Digits: 3
Upper-case: 1
Symbols: 0
Score: 2 out of 5
```

```
Rating: Medium
Letters only: Pass
Digits only: 123
Swapped case: pASS123
Suggestion: P@ss123!
Email: robot@mail.com
Email OK
User: robot
Domain: mail.com
```

- שאלה לכיתה: אילו חלקים בפלט אנחנו כבר יודעים לבנות?

שישה שלבים

שלב	מה מוסיפים	מה לומדים
1	סיסמה ודפוס חלש	`lower` , `startswith` , `endswith`
2	ספירת סוגי תווים	`for ch in text` , `isalpha` , `isdigit`
3	ציון ודירוג	`+= 1` , `if / elif / else`
4	מחרוזות חדשות	בנייה עם `+=`
5	הצעה לסיסמה	`elif` בתוך לולאה
6	כתובת דוא״ל	`count` , `find` , חיתוך

שאלות של כן ולא על מחרוזת

```
text = "Pass123"
small = text.lower()
print(text.startswith("pass"))
print(small.startswith("pass"))
print(text.endswith("123"))
print("7".isdigit())
print("a7".isdigit())
print("x".isalpha())
```

```
False
True
True
True
False
True
```

- `lower()` ו- `upper()` נותנות מחרוזת חדשה. המקורית לא משתנה.
- אותיות גדולות וקטנות הן תווים שונים.

חוזים: כן או לא

⏱ 3 דקות

חזו על הנייר את חמש שורות הפלט.

```
text = "Go Pro"
print(text.upper())
print(text.lower().startswith("go"))
print(text.endswith("PRO"))
print("2027".isdigit())
print("20.27".isdigit())
```

חוזים: התשובה

```
GO PRO
True
False
True
False
```

- `endswith("PRO")` שואלת על אותיות גדולות, ולכן `False`.
- הנקודה אינה ספרה, ולכן `"20.27".isdigit()` היא `False`.

שלב 1: סיסמה ודפוס חלש

```
password = input("Password: ")
while len(password) < 6:
    print("Too short, try again")
    password = input("Password: ")
lowered = password.lower()
weak = False
if lowered.startswith("pass"):
    weak = True
if lowered.endswith("123"):
    weak = True
if weak:
    print("Weak pattern found")
```

```
Password: abc
Too short, try again
Password: Pass123
Weak pattern found
```

- למה בודקים על `lowered` ולא על `password`?
- מה זוכר המשתנה `weak`?

עוברים על תווים בלולאה

```
for ch in "abc":
    print(ch, end=",")
print()
```

```
a,b,c,
```

- בכל סיבוב `ch` הוא תו אחד של המחרוזת.
- `end=","` משאיר את ההדפסה הבאה באותה שורה.

חוזים: הלולאה

⏱ 3 דקות

מה יודפס? כתבו על הנייר.

```
text = "Hi 5 you2"
total = 0
for ch in text:
    if ch.isdigit():
        total += 1
print(total)
```

חוזים: התשובה

2

- `isdigit()` נבדקת על כל תו בנפרד.
- המונה מתחיל ב-`0` לפני הלולאה ומודפס אחריה.

שלב 2: סופרים סוגי תווים

```
letters = 0
digits = 0
upper = 0
symbols = 0
for ch in password:
    if ch.isalpha():
        letters += 1
        if ch == ch.upper():
            upper += 1
    elif ch.isdigit():
        digits += 1
    else:
        symbols += 1
```

```
Letters: 4
Digits: 3
Upper-case: 1
Symbols: 0
```

- למה `upper` נספר רק בתוך `isalpha`?

שלב 3: ציון ודירוג

```
score = 0
if len(password) >= 8:
    score += 1
if digits > 0:
    score += 1
if upper > 0:
    score += 1
if symbols > 0:
    score += 1
if not weak:
    score += 1
```

```
Score: 2 out of 5
Rating: Medium
```

- 4 ומעלה: חזקה. 2 ומעלה: בינונית. אחרת: חלשה.
- איך נכתוב את הדירוג עם `if / elif / else`?

שלב 4: בונים מחרוזות חדשות

```
only_letters = ""
only_digits = ""
swapped = ""
for ch in password:
    if ch.isalpha():
        only_letters += ch
    if ch.isdigit():
        only_digits += ch
    if ch == ch.upper():
        swapped += ch.lower()
    else:
        swapped += ch.upper()
```

```
Letters only: Pass
Digits only: 123
Swapped case: pASS123
```

- מחרוזת אי אפשר לשנות במקום, לכן בונים חדשה עם `+=`.

שלב 5: הצעה לסיסמה חזקה

```
suggestion = ""
for ch in password:
    small = ch.lower()
    if small == "a":
        suggestion += "@"
    elif small == "e":
        suggestion += "3"
    elif small == "o":
        suggestion += "0"
    else:
        suggestion += ch
if not suggestion.endswith("!"):
    suggestion += "!"
```

```
Suggestion: P@ss123!
```

- למה משווים ל-`small` ולא ל-`ch`?
- למה בודקים `endswith("!")`?

שלב 6: כתובת דוא"ל

```
email = input("Email: ")
at = email.find("@")
good = email.count("@") == 1
if good and not email[0].isalpha():
    good = False
com = email.endswith(".com")
il = email.endswith(".co.il")
if good and not (com or il):
    good = False
if good:
    print("Email OK")
    print("User:", email[:at])
    print("Domain:", email[at + 1:])
else:
    print("Email not valid")
```

```
Email: robot@mail.com
Email OK
User: robot
Domain: mail.com
```

- למה `good and` לפני `email[0]`?

שגיאות שכדאי להכיר

שימוש שגוי

```
count = 0
for ch in "sometimes":
    count = 0
    if ch == "e":
        count += 1
print(count)
```

```
0
```

שימוש שגוי

```
email = ""
print(email[0])
```

```
IndexError: string index out of range
```

- המונה מתאפס בכל סיבוב, לכן מאתחלים לפני הלולאה.

מה למדנו

כלי	מה הוא עושה	דוגמה
`lower`	אותיות קטנות	`"Go".lower()`
`upper`	אותיות גדולות	`"Go".upper()`
`startswith`	כן או לא: התחלה	`"Pass1".startswith("Pa")`
`isdigit`	כן או לא: ספרה	`"7".isdigit()`
`for ch in text`	עוברים על התווים	`for ch in "abc"`
`+=`	מונה או בניית מחרוזת	`count += 1`

בדיקת יציאה

⏱ 2 דקות

מה הערך?

```
text = "Go2Ro"
```

1. `text.lower()`
2. `text[2].isdigit()`
3. `text.endswith("ro")`
4. `text.count("o")`

בדיקת יציאה: התשובות

```
text.lower()              go2ro
text[2].isdigit()         True
text.endswith("ro")       False
text.count("o")           2
```

- `endswith("ro")` לא מוצאת `Ro`: אות גדולה היא תו אחר.
- `count` סופרת רק `o` קטנה.